THE HONG KONG POLYTECHNIC UNIVERSITY
香港理工大學

COMP4442 Service and Cloud Computing
Tutorial 6

LI Yuhui
merrill.li@connect.polyu.hk

March 4, 2026

Trends of Cloud Computing and Beyond

- Cloud Computing to Cloud Native
- From Cloud Computing to Edge Computing and Edge Native
- Recent Trend of AI Native
- Future

Cloud Native

Cloud Native

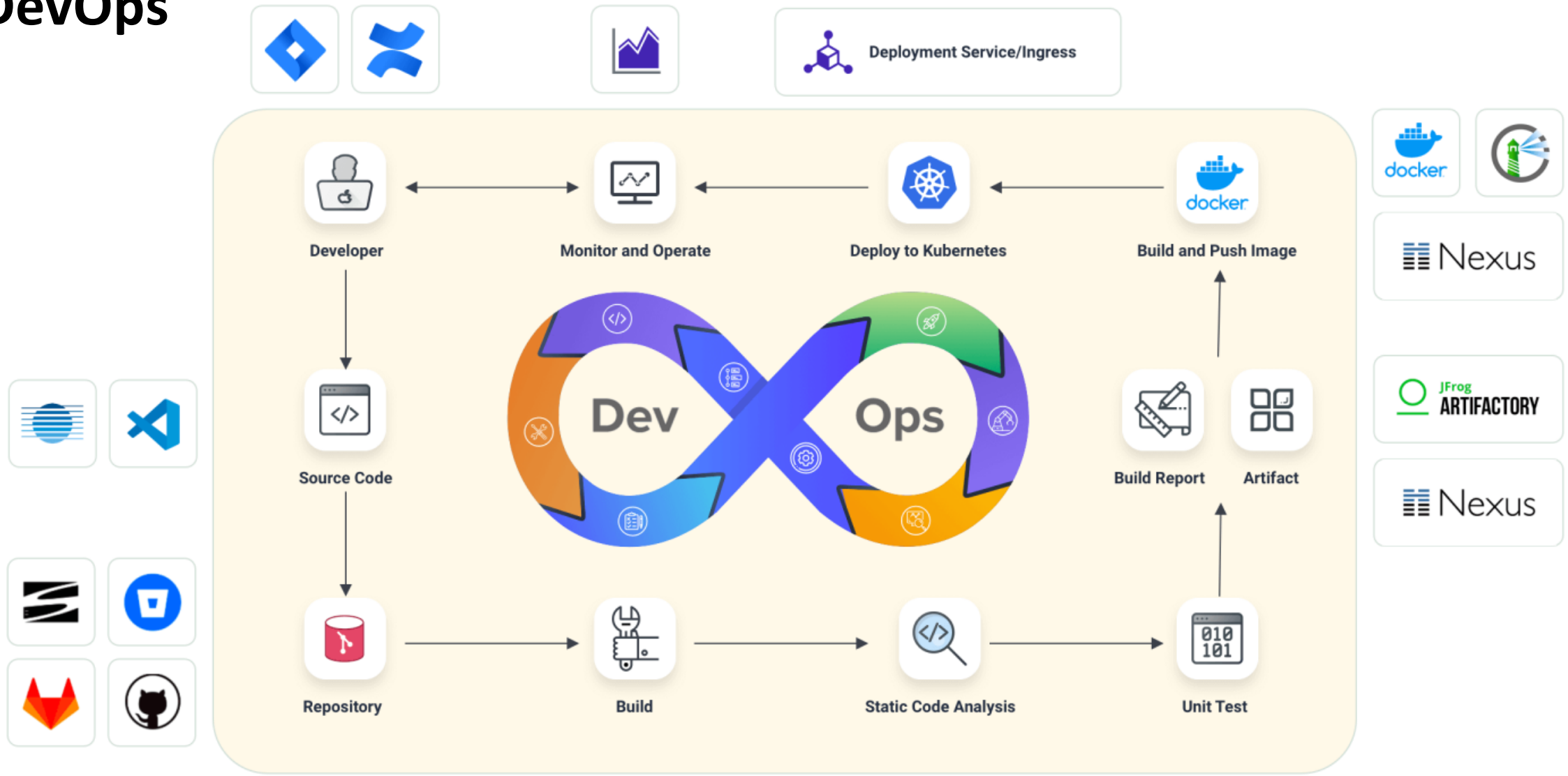
- **Cloud native computing** is an approach in software development that utilizes cloud computing to “build and run scalable applications in modern, dynamic environments such as public, private, and hybrid clouds”. These technologies, such as containers, microservices, serverless functions, cloud native processors and immutable infrastructure, deployed via declarative code are common elements of this architectural style. Cloud native technologies focus on minimizing users' operational burden.

- **Keywords:** Containerization, Orchestration, Microservices, DevOps&CI/CD

Cloud Native: DevOps

- **What is CI/CD?**
  - CI/CD stands for Continuous Integration and Continuous Deployment (or Continuous Delivery). It's a set of practices and tools designed to improve the software development process by automating builds, testing, and deployment, enabling you to ship code changes faster and reliably.
  - **Continuous integration (CI):** Automatically builds, tests, and integrates code changes within a shared repository
  - **Continuous delivery (CD):** automatically delivers code changes to production-ready environments for approval
  - **Continuous deployment (CD):** automatically deploys code changes to customers directly

DevOps


Deployment Service/Ingress
Developer
Monitor and Operate
Deploy to Kubernetes
Build and Push Image
Source Code
Dev
Ops
Build Report
Artifact
Repository
Build
Static Code Analysis
Unit Test
docker
Nexus
JFrog ARTIFACTORY
Nexus
maven
Jenkins
sonarqube
JUnit

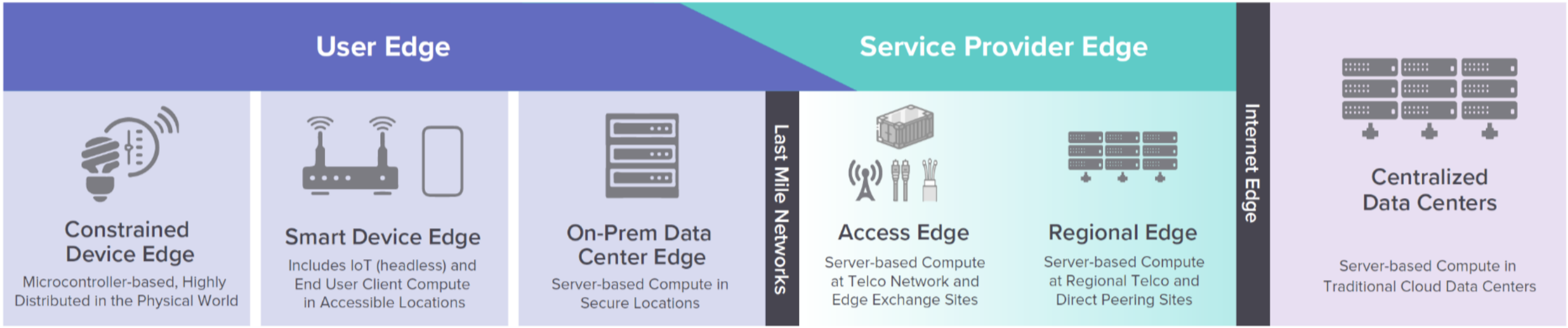
Cloud to Edge, and Edge Native

Edge Native

- **Edge computing** is a distributed computing model that brings computation and data storage closer to the sources of data. More broadly, it refers to any design that pushes computation physically closer to a user, so as to reduce the latency compared to when an application runs on a centralized data centre.

- The term began being used in the 1990s to describe content delivery networks—these were used to deliver website and video content from servers located near users.[2] In the early 2000s, these systems expanded their scope to hosting other applications, leading to early edge computing services. These services could do things like find dealers, manage shopping carts, gather real-time data, and place ads.

Edge Native


User Edge
Service Provider Edge
Constrained Device Edge
Microcontroller-based, Highly Distributed in the Physical World
Smart Device Edge
Includes IoT (headless) and End User Client Compute in Accessible Locations
On-Prem Data Center Edge
Server-based Compute in Secure Locations
Last Mile Networks
Access Edge
Server-based Compute at Telco Network and Edge Exchange Sites
Regional Edge
Server-based Compute at Regional Telco and Direct Peering Sites
Internet Edge
Centralized Data Centers
Server-based Compute in Traditional Cloud Data Centers

Edge Computing: New Challenges Introduced

- **Network Reliability & Latency**
  - Decentralized infrastructure may face unstable connections and inconsistent latency.
  - Requires dynamic routing and fault tolerance strategies.

- **Resource-Constrained Devices**
  - Limited compute, storage, and power at edge nodes.
  - Demands lightweight algorithms and efficient resource orchestration.

- **Scalability & Heterogeneity**
  - Managing diverse hardware/software ecosystems (IoT sensors, gateways, etc.).
  - Scaling dynamically while maintaining performance consistency.

Difference?

- **Solution to Network Reliability & Latency**
  - Edge Autonomy. (See KubeEdge)
  - KubeEdge: Plugin for Kubernetes to manage edge nodes. (https://kubeedge.io/)

- **Resource-Constrained Devices**
  - Lightweight implementations for some key components.
  - Message Queues, k3s...

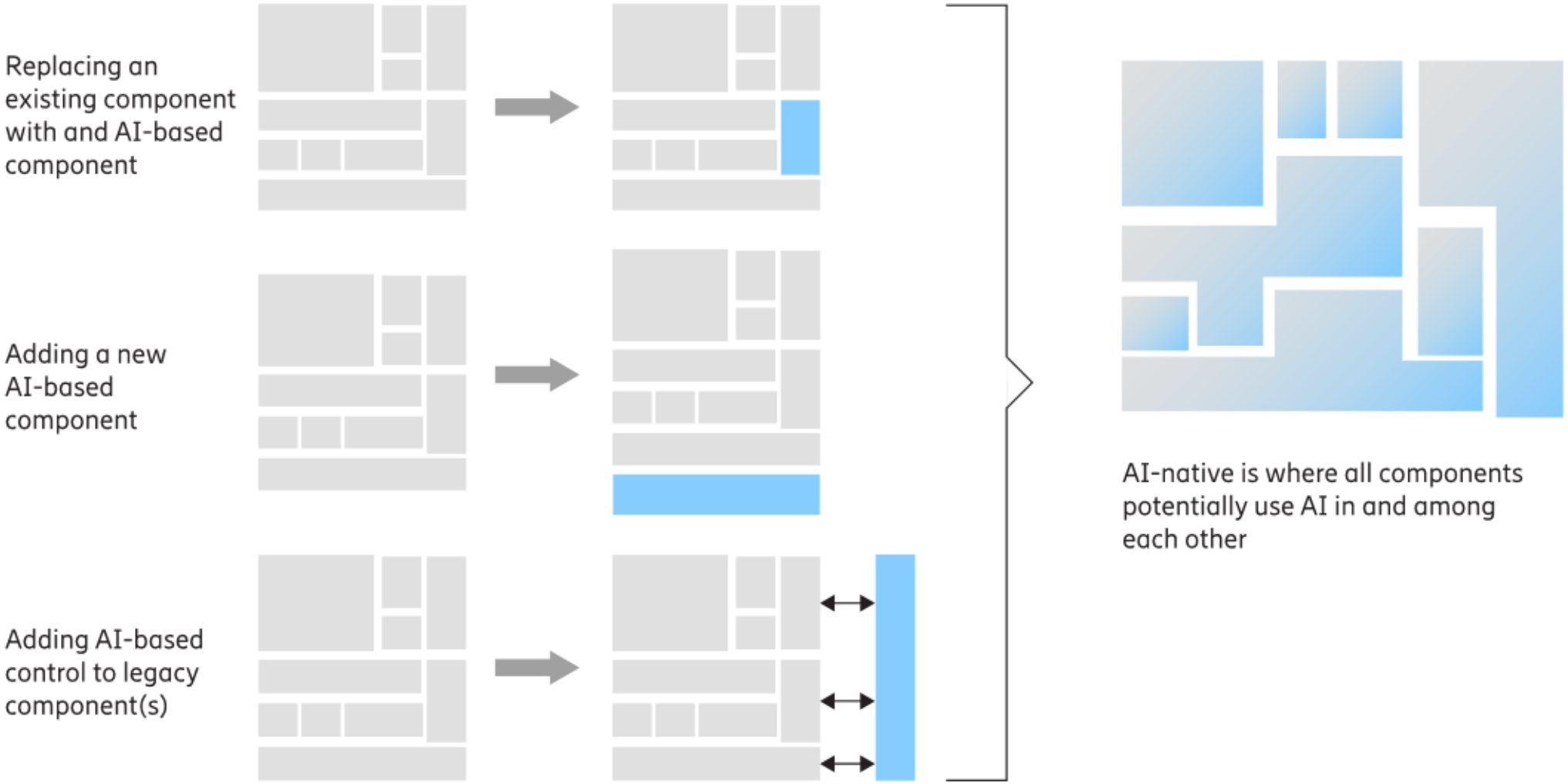
- **Scalability & Heterogeneity**
  - Extra configuration files to abstract the hardware capability.
  - Include sensor into kubernetes management.

AI Native

AI Native

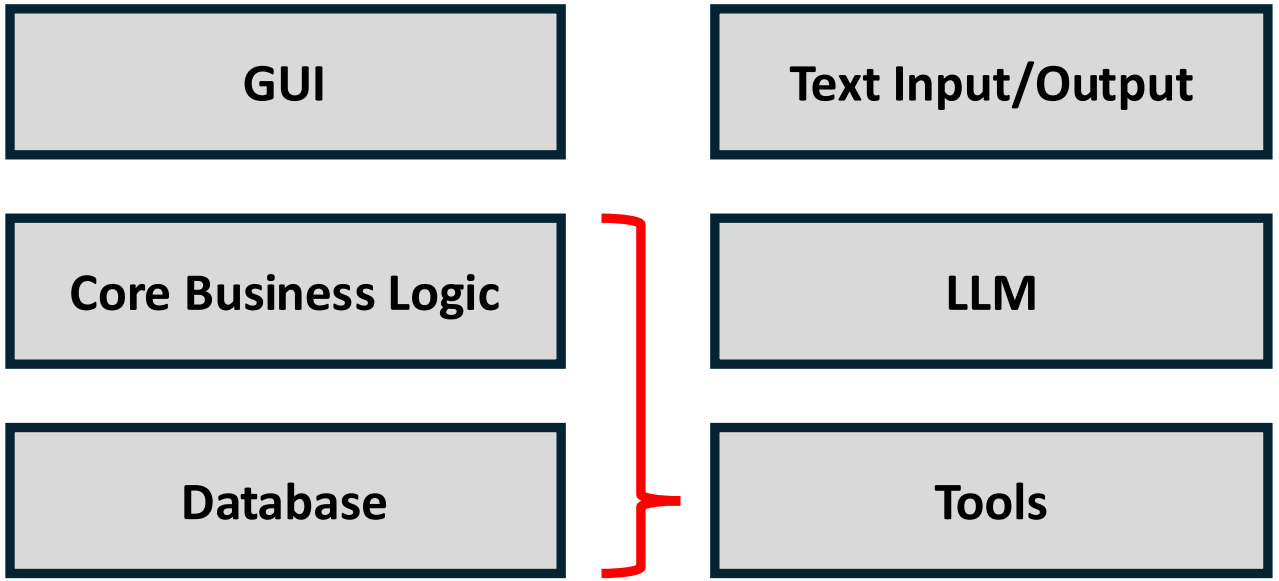
- **What is AI-native?**
  - AI-Native refers to systems, applications, or services designed from inception with artificial intelligence (AI) as their foundational driver.
    - Not merely "adding AI" as a feature, but embedding AI into the core architecture.
    - AI governs the entire lifecycle: design, development, and operation.
  - Represents the shift from AI as a "tool" (e.g., plugins) to AI as a **fundamental layer** of digital infrastructure.
  - AI native is often used as a prefix to an entity like a system, function, or implementation.
    - Pervasive use of AI and a required accompanying data infrastructure in all sub-components of an entity rather than adding on an AI-based component to an existing non-AI-based entity.

AI Native

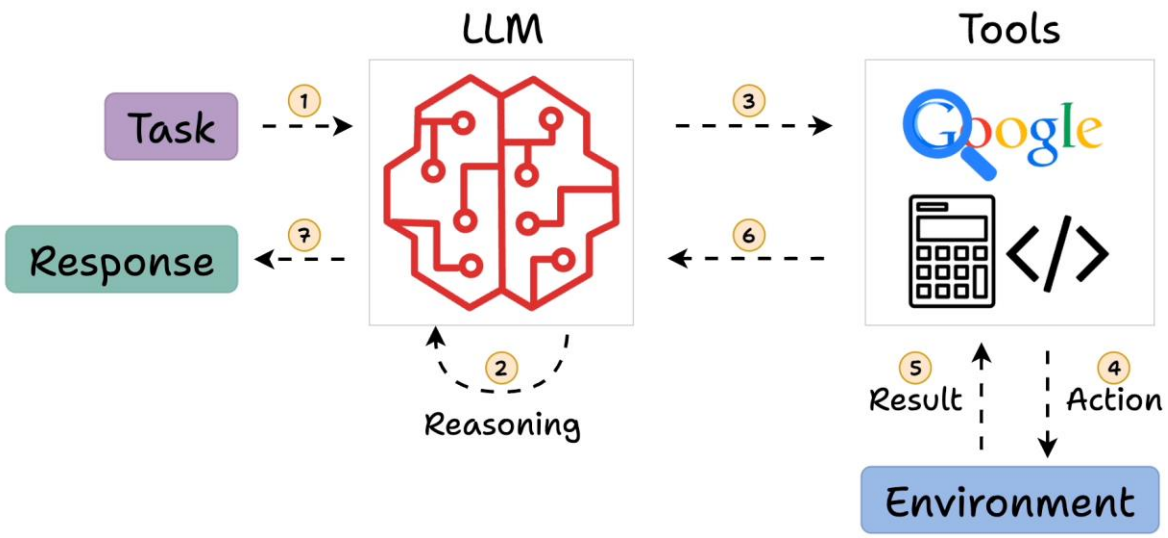

Replacing an existing component with and AI-based component
Adding a new AI-based component
Adding AI-based control to legacy component(s)
AI-native is where all components potentially use AI in and among each other

AI Native

- A software architecture stack that has been turned upside down. Instead of a traditional database at the bottom and UI at the top, a glowing, multi-layered neural network sits right in the middle as the core engine. Surrounding it are interconnected nodes labeled Vector Database, RAG, and Tools. The user interface shows a simple chat bubble instead of complex menus. The key visual message is: The model is the application.



(a) Legacy Software (b) AI Software



Cloud Native AI

CNAI

Data Architecture

ClickHouse, druid, cassandra, SCYLLA, presto, trino, Spark, Kubeflow, Flink, kafka, PULSAR, redis, ALLUXIO, Kubeflow, FEAST, Fluid

Model/LLM Observability

trulens, Okahu, OpenLIT, DeepFlow

General Orchestration

kubernetes — CNCF GRADUATED
VOLCANO — CNCF INCUBATING
RAY, ARMADA, HAMi

CI/CD - Delivery

Kubeflow, mlflow, MLRun

Distributed Training

Kubeflow, PyTorch, PyTorch, deepspeed, Megatron-LM, Alpa, Kubeflow

Workload Observability

Grafana, W&B

Governance, Policy & Security

Kyverno — CNCF INCUBATING
spicedb, PERMIFY, JSON, stacklok

AutoML

HYPEROPT, Katib

ML Serving

vLLM, KServe, vLLM

Vector Databases

Milvus

Data Science

marimo, jupyter, Kubeflow, PyTorch, dmlc XGBoost, Apache Zeppelin

Open Enterprise AI Bluep...

Open Platform for Enterprise AI

How about Cloud/Edge Native?

- They have been compressed as a single layer in the era of AI.
- We spent decades to build the basic infra on the cloud and the edge.
- Now, they have incubated AI.
- AI is the top layer we are focusing on.
  - We build a complex system layer by layer.

OpenClaw

A highly personal, local setup. A sleek laptop or a Mac mini sits on a desk. From this single local machine, glowing lines extend outward to various apps: WhatsApp, Slack, a local file system, and an email inbox. Inside the machine, there is a small, autonomous robot icon representing the OpenClaw agent. A clock symbol indicates it is running 24/7. The key visual message is: Your personal digital worker, operating locally and reaching out globally.

Offical Website: https://openclaw.ai/

OpenClaw

It is important to clarify that OpenClaw does not kill cloud computing. Instead, it changes how individuals interact with the cloud. Your OpenClaw agent lives locally, but it will still reach out to call Cloud Native APIs, like a weather service, a flight booking system, or a massive AI model for tasks too heavy for local hardware. Cloud Native becomes the invisible, standardized backend utility, while OpenClaw becomes the highly personalized frontend executor.

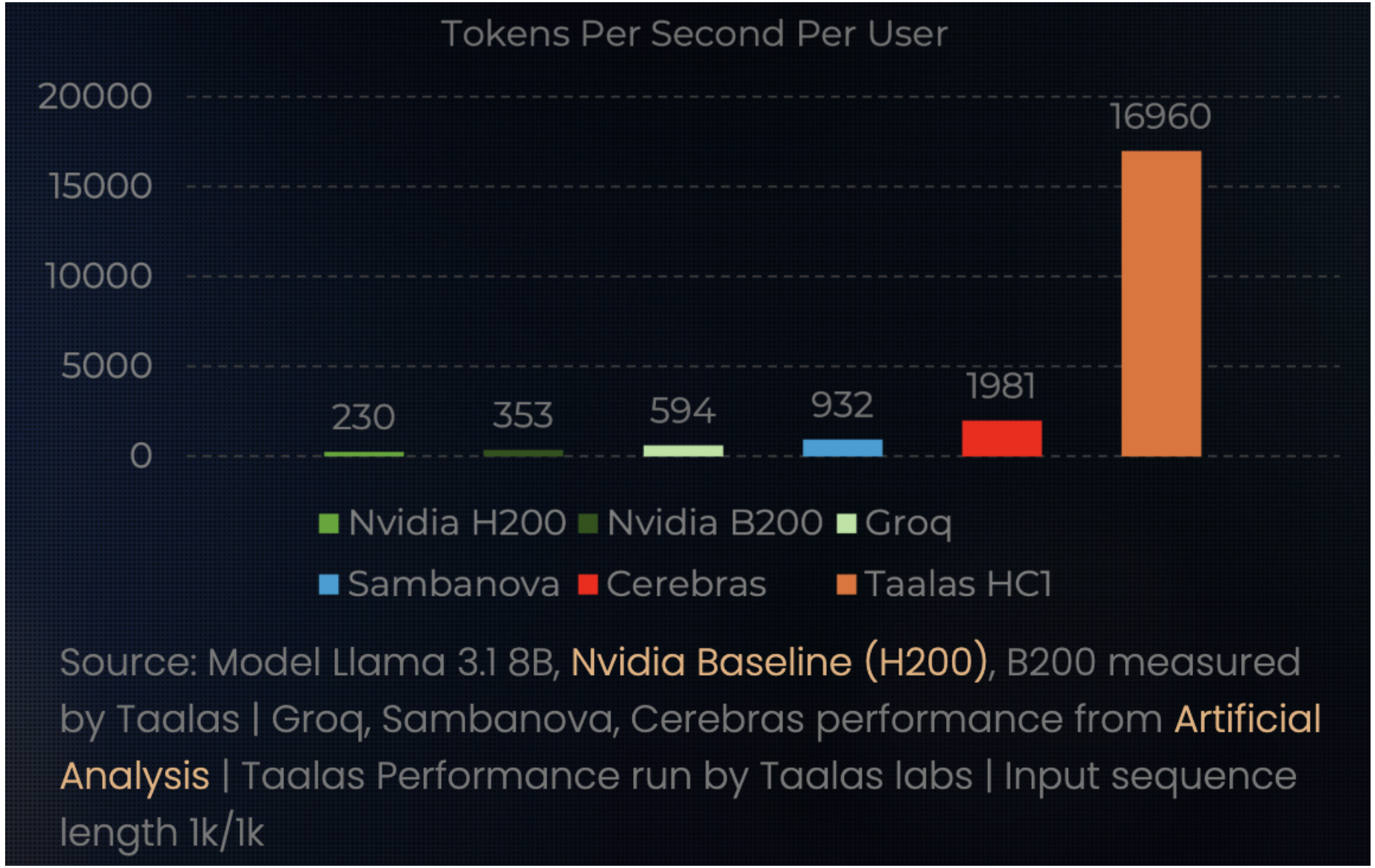
Current Drawbacks of OpenClaw:

- Low efficiency due to LLM inference at 100 token/s.
- High cost of inference cost. $3 for 1M inputs, $15 for 1M outputs
- Every day, I spend 10RMB for automatic tasks by using OpenClaw.

AI as Software Core, AI as OS, AI as Computer!!!

- Model as Silicon:
  - Current GPU/NPU are somewhat general-purpose.
  - How about xPU for LLM only?
  - Taalas HC1 achieves 15000+ token/s for Llama 3 8B.

AI as Software Core, AI as OS, AI as Computer!!!



AI as Software Core, AI as OS, AI as Computer!!!

- Small models are quite good.
  - Only one step left to do: Agentic Ability

- What does it mean for 15000 token/s? If 15000 token/s scale up to 150,000 token/s?
  - Cost reduction for inference. ($3 ---> $0.05 per 1M tokens)
  - Summarize/Translate a long paper?
  - Write a software?
  - Automatic Research?
  - Render your GUI?
  - What is the meaning of human labor?

- What would happen beyond AI as Computer?? AI as a creature?!